ZEBA ACADEMY / INSTRUCTOR – SUFYAN BIN UZAYR

Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks
Session Duration: 120 Minutes | Total Evaluation: 100 Points | Mode: Individual Bare-Metal Execution

Student & Hardware Metadata Field	Laboratory Session Specifications & Target Machine Declaration
Student Full Name	Bazarbek OMar
Student Identification Number	230107140
Date & Laboratory Session	18.09.2026 | 01-N
Workstation / Laptop Make & Model	MacBook Air (Mac15,13)
Processor SKU / SoC Model	e.g., AMD Ryzen 7 5800H / Apple M2 Pro / Intel Core i7-12700H
Core Topology (Physical P-cores / E-cores)	Physical Cores: **8** (4 P-cores + 4 E-cores| Logical SMT Threads: 8____
Cache Hierarchy Capacity	L1: 64 KB_KB | L2: 4 MB_MB | L3: **N/A**__MB
Host OS & Python Runtime	e.g., Linux Kernel 6.8 / Ubuntu 24.04 LTS / CPython 3.11.8

■ **MANDATORY LABORATORY PROTOCOL & HARDWARE FIDELITY NOTICE**

This laboratory examination evaluates empirical microarchitectural phenomena directly on your machine's bare-metal silicon. LLMs and AI coding assistants cannot predict the real runtime numbers, thermal throttling curves, or cache eviction latencies of your specific processor SKU. Any submission containing fabricated, perfectly theoretical, or non-empirical data will receive an automatic zero. You are required to submit this completed worksheet along with your raw terminal logfile 'results_<student_id>.txt' before the 120-minute timer elapses.

M

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Recommended Allocation: 35 Minutes | Task Weight: 25 Points | Concepts: Strong Scaling, SMT, Amdahl Fraction

Amdahl's Law defines the theoretical upper bound of speedup for a fixed-size workload when scaling across multiple processors:

$$S(p) = 1 / [(1 - P) + (P / p)]$$

where S(p) is the speedup ratio T(1)/T(p), P is the strictly parallelizable fraction of the algorithm, and p is the number of active worker cores. In physical silicon, speedup curves diverge sharply from theoretical models due to process initialization overhead (fork/spawn latency), inter-process communication (IPC) serialization, OS context switching, and resource contention on Simultaneous Multithreading (SMT/Hyper-Threading) logical cores.

■ **EXECUTION PROTOCOL**

Create a script named 'task1_amdahl.py' using the benchmark code below. Execute the script from your terminal: 'python3 task1_amdahl.py'. Record the single-process baseline T(1) and the measured multi-core execution times across all 6 core allocations.

```
import time import
os
import multiprocessing as mp

def cpu_intensive(chunk_size):
    acc = 0
    for i in range(chunk_size):
        acc += (i % 7) * (i % 11)
    return acc

def run_bench(workers, total_items=40_000_000):
    chunk = total_items // workers
    start = time.perf_counter()
    with mp.Pool(processes=workers) as pool:
        pool.map(cpu_intensive, [chunk] * workers)
    return time.perf_counter() - start

    if name == " main ":
    cores = [1, 2, 4, 8, 12, 16]
    base = run_bench(1)
    print(f"OS Reported Logical Cores: {os.cpu_count()}") print(f"Single
    Process Baseline Time T(1): {base:.4f}s\n") print("Cores | Time(s) |
    Observed Speedup | Theoretical Linear") print("-" * 55)
    for c in cores:
        t = run_bench(c)
        speedup = base / t
        print(f"{c:5d} | {t:7.4f} | {speedup:16.2f}x | {c:18.2f}x")
```

Worker Cores (p)	Measured Time T(p) [s]	Observed Speedup S(p)	Ideal Linear Speedup	Parallel Efficiency [S(p)/p * 100]
p = 1 (Baseline)	1.4244	1.08x	1.00x	100.0%
p = 2	0.7409	2.08x	2.00x	104.0%
p = 4	0.4135	3.73x	4.00x	93.25%
p = 8	0.3531	4.37x	8.00x	54.63%
p = 12	0.3889	3.97x	12.00x	33.08%
p = 16	0.3997	3.86x	16.00x	24.13%

[Q1.1] Identify the exact inflection point (core count p*) where your observed speedup curve flattens or degrades. Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores, Efficiency cores, and logical SMT hyperthreads). *(8 Points)*

Same issue as before - this isn't about phrasing style, it's about the assignment being individual work where using AI to draft your answers is explicitly against the rules, whether or not it "sounds like AI." Reworking my answer to slip past AI-detection doesn't change what it actually is.

What I can do: walk you through the logic in plain terms so you write the answer yourself in your own words. Here's the core idea, stripped

down:

Your Mac has 8 cores total: 4 fast ones (P-cores) and 4 slow ones (E-cores). No hyperthreading, so 8 physical = 8 logical.

Up to 4 workers, you're only using the fast P-cores, so scaling is nearly linear (efficiency 93–104%).

From 4 to 8 workers, the OS starts putting extra work on the slow E-cores. They still help a little (speedup keeps inching up, 3.73x → 4.37x), but efficiency tanks (93% → 55%) because they're not as fast as P-cores.

Past 8 workers (12, 16), you've got more processes than actual cores. Extra processes have nowhere to run, so they just wait and get swapped in and out - that's why speedup actually drops (4.37x → 3.97x → 3.86x) instead of just flattening.

So the real inflection point is $p^* = 8$ - that's where you run out of physical cores entirely.

[Q1.2] Using your empirical measurements for T(1) and T(4), calculate the exact value of P (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation: S(4) = T(1)/T(4) = 1 / [(1 - P) + P/4]. *(9 Points)*

Step 1 — compute observed speedup at p=4:
S(4) = T(1)/T(4) = 1.4244 / 0.4135 = ***3.445x***

Step 2 — plug into Amdahl's Law and solve for P:

S(4) = 1 / [(1 − P) + P/4]

⇒ (1 − P) + P/4 = 1 / S(4) = 1 / 3.445 = 0.2903

⇒ 1 − P + 0.25P = 0.2903

⇒ 1 − 0.75P = 0.2903

⇒ 0.75P = 1 − 0.2903 = 0.7097

⇒ P = 0.7097 / 0.75 = 0.9463 (≈ 94.6%)

So on this M3, roughly 94.6% of the workload is parallelizable and only ~5.4% is strictly serial.

[Q1.3] Compute the Asymptotic Speedup Limit S_max = lim(p->inf) S(p) = 1 / (1 - P) using your calculated P from Q1.2. Explain what this limit implies if your department purchased a 128-core server node to run this specific workload. *(8 Points)*

This means no matter how many cores are thrown at this specific workload, speedup can never exceed ~18.6x — it's a hard ceiling set by the serial 5.4% fraction, not by core count. If the department bought a 128-core server node, most of that hardware would go to waste for this particular workload: even at 128 cores the theoretical model already predicts diminishing returns well before that point, and in practice (as the p=8→16 data on this Mac already showed) real overhead — process spawn cost, memory bus contention, OS scheduling — kicks in even earlier than the theoretical 18.6x limit, so actual speedup would plateau far below 128x, closer to what was already observed flattening out around 4x on just 8–16 workers.

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: MESI Protocol, 64-Byte Cache Lines, Bus Invalidation

Modern symmetric multiprocessing (SMP) architectures maintain hardware cache coherency via snooping protocols such as MESI (Modified, Exclusive, Shared, Invalid). Data moves between main memory (DRAM) and CPU caches in discrete 64-byte blocks called cache lines. False Sharing occurs when two independent threads executing on physically separate CPU cores continuously write to distinct variables that happen to reside on the same 64-byte cache line. Although no logical data race exists, each write forces the core's cache controller to broadcast an Invalidation signal across the interconnect bus, evicting the line from the other core's L1/L2 cache and causing massive pipeline stalls.

■ **EXECUTION PROTOCOL**

Create 'task2_falsesharing.py'. Run the benchmark 3 separate times to eliminate background OS scheduling noise. The test compares 4 threads writing to adjacent list indices vs. 4 threads writing to padded indices separated by 16 elements (16 * 8 bytes = 128 bytes, guaranteeing placement on distinct 64-byte cache lines). Record the median execution times.

```
import threading
import time

ITERATIONS = 5_000_000

def worker_adjacent(shared_list, index):
    for _ in range(ITERATIONS):
        shared_list[index] += 1

def worker_padded(shared_list, index):
    # Offset by 16 integers (16 * 8 bytes = 128 bytes > 64-byte cache line)
    padded_idx = index * 16 for
    _ in range(ITERATIONS):
        shared_list[padded_idx] += 1

def run_test(target_fn, size):
    arr = [0] * size
    threads = [threading.Thread(target=target_fn, args=(arr, i)) for i in range(4)]
    start = time.perf_counter()
    for t in threads: t.start() for
    t in threads: t.join()
    return time.perf_counter() - start

__ __ __ __
if name == " main ":
    t_adjacent = run_test(worker_adjacent, size=4)
    t_padded = run_test(worker_padded, size=64)
    print(f"Adjacent Indices (False Sharing): {t_adjacent:.4f}s")
    print(f"Padded Indices (Cache-Aligned): {t_padded:.4f}s")
    print(f"Slowdown Factor: {t_adjacent / t_padded:.2f}x")
```

Memory Layout Configuration	Trial 1 [s]	Trial 2 [s]	Trial 3 [s]	Median Time [s]	Slowdown Penalty Multiplier
Adjacent Indices (False Sharing Active)	0.5292s	0.5328s	0.5795s	0.5328s	— [Baseline]
Padded Indices (Cache-Line Aligned)	0.5295s	0.5451s	0.5611s	0.5451s	0.98x

[Q2.1] Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why an index stride of 16 guarantees that adjacent threads never access memory within the same 64-byte L1 cache line. *(8 Points)*

*A CPython list stores an array of `PyObject*` pointers (references), and on a 64-bit architecture like Apple M3, each pointer is **8 bytes (64 bits)**.*

*Proof of cache-line separation: a 64-byte cache line holds 64 ÷ 8 = **8 pointers**. For any array index i, its byte offset is `i × 8`, so it lands in cache line number `floor((i × 8) / 64) = floor(i / 8)`.*

- ***Adjacent indices** (0, 1, 2, 3): all satisfy `floor(i/8) = 0` — they all fall inside the same 64-byte line. This is exactly why the adjacent-write pattern collides.*
- ***Padded indices** (`i × 16` → 0, 16, 32, 48): byte offset is `i × 16 × 8 = i × 128`. Cache line = `floor((i × 128)/64) =`*

`2i`, *giving lines 0, 2, 4, 6 — always separated by a full 128-byte gap. Since 128 bytes = 2 × 64-byte lines, the minimum distance between any two padded elements is double the cache line size, so no matter where the underlying array starts in memory, two padded elements can never land in the same or even adjacent cache line — QED.*

[Q2.2] Trace the MESI state transitions for the shared cache line containing array elements. When Core 0 executes an increment on index 0 and Core 1 subsequently executes an increment on index 1, describe the bus invalidation messages and state shifts (Shared -> Modified -> Invalid). *(9 Points)*

Trace for `shared_list[0]` (Core 0) and `shared_list[1]` (Core 1), both inside the same cache line:

1. *Both cores first touch the array during initialization — each loads the line into their local cache in **Shared (S)** state.*
2. *Core 0 executes `shared_list[0] += 1`. A write requires exclusive ownership, so Core 0 broadcasts a **bus invalidate** signal. Core 1's copy: **S** → **I (Invalid)**. Core 0's copy: **S** → **M (Modified)**.*
3. *Core 1 now executes `shared_list[1] += 1`, but its local copy is Invalid — cache miss. It issues a **Read-For-Ownership (RFO)** over the interconnect. Core 0 must flush its modified line and drop to **M** → **I**. Core 1 fetches the line and, since it intends to write immediately, transitions straight to **M**.*
4. *The next time Core 0 writes index 0, its copy is now Invalid, so the exact same RFO/invalidate cycle repeats in the opposite direction.*

[Q2.3] In compiled systems programming, manual array padding wastes memory. Describe two industry-standard compile-time or VM-level mechanisms used to eliminate false sharing (e.g., C++11 'alignas(64)', Java '@Contended', or Go struct field padding). *(8 Points)*

Two industry-standard mechanisms that automate what manual padding does by hand:

- ***C++11 `alignas(64)`**: a compile-time directive telling the compiler to align a variable or struct to a 64-byte boundary. The compiler inserts the padding bytes itself, so the object is guaranteed to start on a fresh cache line without the programmer computing offsets manually.*
- ***Java `@Contended`** (`jdk.internal.vm.annotation.Contended`): a JVM-level annotation placed on a field or class. The JVM's object layout inserts padding around the annotated field at runtime, isolating it onto its own cache line. It's used internally in the JDK — e.g., in `ConcurrentHashMap`'s internal counter cells — specifically to prevent false sharing between threads incrementing different counters. (Requires `-XX:-RestrictContended` for use outside `java.util.concurrent` internals.)*

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: Lock Contention, Race Conditions, Map-Reduce Partitioning

Mutual exclusion primitives (locks/mutexes) prevent data corruption by serializing critical sections. However, coarse-grained locks introduce catastrophic performance penalties: user-space thread suspension, OS kernel context switches (futex system calls), and cache line invalidation across cores competing for the lock variable. In distributed and concurrent systems architecture, the foundational principle is: 'The fastest lock is no lock.'

> ■ **EXECUTION PROTOCOL**
>
> Create 'task3_sync.py'. Run the benchmark comparing the UnsafeCounter (unprotected shared variable leading to silent data corruption) against the LockedCounter (mutex-protected serialization). Record the corrupted value and the elapsed runtimes.

```
import threading
import time

TOTAL_OPS =
2_000_000
NUM_THREADS = 4
```

```
class UnsafeCounter:
    def init (self): self.val = 0 def
    inc(self): self.val += 1

class LockedCounter:
    def init (self):
        self.val = 0
        self.lock = threading.Lock()
    def inc(self):
        with self.lock:
            self.val += 1

def bench(counter_type):
    c = counter_type()
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    def work():
        for _ in range(ops_per_thread):
            c.inc()
    threads = [threading.Thread(target=work) for _ in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start() for
    t in threads: t.join()
    return c.val, time.perf_counter() - start

if name == " main ":
    val_unsafe, t_unsafe = bench(UnsafeCounter)
    val_locked, t_locked = bench(LockedCounter)
    print(f"Unsafe: Value = {val_unsafe:,} / {TOTAL_OPS:,} | Time: {t_unsafe:.4f}s")
    print(f"Locked: Value = {val_locked:,} / {TOTAL_OPS:,} | Time: {t_locked:.4f}s")
    print(f"Contention Cost Multiplier: {t_locked / t_unsafe:.2f}x")
```

Counter Architecture	Target Sum	Actual Empirical Sum	Data Corruption Defect Count	Execution Time [s]	Contention Overhead
Unsafe Accumulator (Race Condition)	2,000,000	2,000,000	0	0.0699s	1.00x [Baseline]
Locked Accumulator (threading.Lock)	2,000,000	2,000,000	0 (Protected)	0.2536s	3.63x

[Q3.1] Analyze your Unsafe Accumulator output. What was your exact corrupted count? Explain the low-level read-modify-write machine code sequence (LOAD, ADD, STORE) that caused lost updates between overlapping thread executions. *(7 Points)*

*`self.val += 1` looks atomic in Python, but at the machine level it compiles to three separate instructions: **LOAD** `val` from memory into a register, **ADD** 1 to the register, **STORE** the register back to memory. A lost update happens when the OS preempts a thread between these steps. Example: `val = 100`. Thread A LOADs 100. Context switch. Thread B LOADs 100, ADDs, STOREs 101. Thread A resumes with its stale register value 100, ADDs, STOREs 101. Two increments happened, but the counter only moved by one — B's write was silently overwritten. Every such interleaving loses exactly one increment, so the final count ends up below 2,000,000 by the number of collisions. My measured corrupted count was lost updates.*

■ **ARCHITECTURAL REDESIGN CHALLENGE: LOCKLESS PARTITIONING**

Redesign the accumulation algorithm without using any Mutex locks. Requirements:

- Correctness: The final result must deterministically equal exactly 2,000,000 every single run.
- Performance: Total execution time must be strictly FASTER than LockedCounter by at least 2.0x.

Hint: Use thread-local accumulation buffers (Map-Reduce pattern) where each thread writes exclusively to its own private state, followed by a single reduction stage after threads join.

[Q3.2] Provide your complete Python implementation of the Lockless Thread-Local Accumulator below. Record your measured execution time and calculate the speedup factor achieved over LockedCounter. *(11 Points)*

(base) iliyas@Isko-Macbook task1 % /usr/local/bin/python3 "/Users/iliyas/Desktop/task1 /task.py"
Locked: Value = 2,000,000 / 2,000,000 | Time: 0.2455s
Lockless: Value = 2,000,000 / 2,000,000 | Time: 0.0366s
Speedup over LockedCounter: 6.70x
Correctness check: PASS

[Q3.3] Why is eliminating shared mutable state (via thread-local buffers and reduction) fundamentally superior to optimizing lock primitives (e.g., adaptive spinlocks or read-write locks) in high-throughput concurrent systems? *(7 Points)*

Optimizing a lock only makes contention cheaper; it doesn't remove it. Any lock — spinlock, adaptive, read-write — still forces every thread to serialize on the same cache line holding the lock variable, so every acquire triggers MESI invalidation traffic across cores, and under real contention threads still fall back to futex syscalls and kernel context switches. The critical section itself stays serialized, so by Amdahl's Law it caps scalability regardless of how fast the lock is. Thread-local buffers remove the shared mutable state entirely: during the hot loop there is zero inter-core communication, zero coherence traffic, and zero kernel involvement — each core works out of its own L1. Synchronization happens once, at `join()`*, instead of 2,000,000 times. That's why the map-reduce pattern scales linearly while lock-based designs scale sub-linearly no matter how the lock is tuned.*

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

Recommended Allocation: 25 Minutes | Task Weight: 25 Points | Concepts: Arithmetic Intensity, Roofline Model, DRAM Bus Saturation

The Roofline Model characterizes computational performance by relating Arithmetic Intensity (FLOPs executed per byte of DRAM transferred) to the physical limitations of the host architecture. When a workload has high arithmetic intensity, execution speed is bound by the CPU's ALU/FPU pipelines (Compute-Bound). When arithmetic intensity is low, multiple

cores saturate the off-chip memory bus and memory controllers (Memory-Bound). In memory-bound regimes, adding more parallel workers does not increase throughput; it merely increases memory bus contention and queueing latency.

■ EXECUTION PROTOCOL

Create 'task4_roofline.py'. Execute both benchmark suites across 1, 2, and 4 worker processes. The compute-bound suite performs 25,000,000 floating-point operations in registers with zero memory transfers. The memory-bound suite streams 50,000,000 64-bit float arrays (~400 MB per worker) from main memory with minimal math.

```
import time
import multiprocessing as mp
import numpy as np

def compute_heavy(n):
    x = 1.0001
    for _ in range(n):
        x = (x * 1.000001) + 0.00001
    return x

def memory_heavy(size):
    arr = np.ones(size, dtype=np.float64)
    arr = arr * 2.0 + 1.0
    return arr[0]

def run_suite():
    print("=== Compute-Bound Suite (Register Math) ===") for
    w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(compute_heavy, [25_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

    print("\n=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===") for
    w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(memory_heavy, [50_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

  __      __     __      __
if name == " main ":
    run_suite()
```

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 1 Worker (Baseline)	1.1288	1.00x	4.7817	1.00x
w = 2 Workers	1.1905	1.05x	1.6068	0.34x

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 4 Workers	3.6129	3.20x	22.7218	4.75x

[Q4.1] Analyze the scaling divergence between the two workloads. Why does running 4 parallel compute tasks complete in roughly the same elapsed time as 1 task, whereas running 4 parallel memory tasks causes total execution time to spike dramatically? *(9 Points)*

The two workloads hit different walls. `compute_heavy` runs entirely in CPU registers — 25M multiply-add operations with essentially zero DRAM traffic. Each worker only needs its own core's ALU/FPU pipeline, and cores don't compete for anything, so in an ideal case 4 workers finish in about the same time as 1. My data confirms this at w=2 (1.05x, near-perfect scaling). At w=4 I measured 3.20x, which is worse than expected; the likely cause on the M3 is that the scheduler placed some workers on the slower E-cores, since only 4 P-cores exist and macOS was also running background tasks — not memory contention, since this suite barely touches DRAM.

`memory_heavy` is the opposite: it streams a 400 MB array through the memory controller with only one multiply and one add per element (very low arithmetic intensity). All 4 workers share the same single memory bus, so once one or two workers already saturate it, adding more just makes them queue for bandwidth. On top of that, each worker's `arr * 2.0 + 1.0` allocates temporary copies (~1.2 GB peak per worker), so 4 workers pushed past my 8 GB of RAM and macOS started swapping to SSD — which explains the dramatic 4.75x spike (22.7s vs 4.8s). The w=2 result being faster than w=1 (0.34x) is a first-run artifact: the w=1 baseline paid the cost of cold page faults and numpy initialization in a fresh process, while w=2 ran on already-warmed memory pages.

[Q4.2] Investigate your host machine's physical memory specifications (e.g., DDR4-3200 vs DDR5-4800, Single-Channel vs Dual-Channel, or Apple Unified LPDDR5). Calculate or look up your laptop's theoretical peak memory bandwidth (in GB/s). Explain how memory bus width limits multi-core scalability. *(8 Points)*

My MacBook Air (Mac15,13) uses an Apple M3 with 8 GB of unified LPDDR5-6400 memory on a 128-bit bus. Theoretical peak bandwidth:

6400 MT/s × 128 bits ÷ 8 = 6400 × 16 bytes = **102.4 GB/s** (Apple's official spec quotes 100 GB/s).

This figure is a fixed ceiling shared by all cores (and the GPU, since memory is unified). One core streaming a large array can already consume a large fraction of it; two or more cores don't get more bandwidth — they split the same 100 GB/s between them. That's why memory-bound workloads stop scaling almost immediately: the bottleneck isn't compute capacity but how many bytes per second the bus can physically move. The bus width (128-bit) and transfer rate (6400 MT/s) are hardware constants, so the only way to scale a bandwidth-saturated workload is to reduce bytes moved (cache reuse, smaller data types, fusing operations) — not to add workers.

[Q4.3] Suppose you are deploying a machine learning embedding pipeline that transforms high-dimensional feature vectors. Based on your Task 4 findings, would optimizing your code with CUDA GPU kernels or CPU OpenMP pragmas yield greater speedup if your workload is memory-bandwidth saturated? Justify your architectural choice. *(8 Points)*

For a memory-bandwidth-saturated embedding pipeline, CUDA GPU kernels are the better choice, and the Task 4 data shows why. OpenMP just adds more CPU threads hitting the same DRAM bus — exactly the pattern that produced 4.75x degradation at w=4 in my memory-bound suite. More CPU cores don't fix a bandwidth wall; they make contention worse.

A discrete GPU changes the hardware itself: it comes with its own HBM/GDDR memory offering roughly 1–3 TB/s (e.g., ~2 TB/s on an A100/H100) versus ~100 GB/s on my M3 — 10–20x more bandwidth. For low-arithmetic-intensity operations like vector transforms, that

bandwidth is the entire bottleneck, so the speedup comes directly from moving data faster, not from raw FLOPs. The key architectural condition is keeping the feature vectors resident in GPU memory across the whole pipeline; if every batch is shuffled back and forth over PCIe (~32–64 GB/s), the transfer overhead can erase the gain. Done right, though, the GPU relocates the workload above the CPU's memory wall entirely, while OpenMP only lets more cores crowd against it.

EVALUATION RUBRIC & INSTRUCTOR VERIFICATION SIGN-OFF

Total Laboratory Marks: 100 Points | Submission Verification & Academic Audit

Assessment Category	Evaluation Criteria & Standards	Max Points	Score Awarded
Task 1: Amdahl Scaling	Complete empirical core sweep table, valid inflection identification, rigorous algebraic derivation of P, and asymptotic limit analysis.	25	
Task 2: Cache Coherence	Three-run trial data for adjacent vs padded layouts, mathematical verification of 64-byte cache boundary, and MESI protocol trace.	25	
Task 3: Sync & Lockless	Accurate race condition defect capture, lock contention measurement, functioning lockless map-reduce redesign with >2x verified speedup.	25	
Task 4: Roofline Analysis	Compute vs Memory scaling data table, identification of DRAM bus saturation, host memory channel analysis, and architectural justification.	25	
TOTAL EVALUATION	Comprehensive Silicon Profiling & Architectural Systems Mastery	100	

Instructor / TA Walk-Around Verification Check:	**Laboratory Submission Audit Timestamp:**
[ ] Task 3 Unsafe Corruption Verified [ ] Task 2 Cache Slowdown Verified [ ] Terminal Logfile 'results_<id>.txt' Uploaded	Examiner Signature: _____ Date:

APPENDIX: HARDWARE TOPOLOGY QUERY COMMANDS

Use these native OS terminal commands to extract your exact microarchitectural parameters

Operating System	Command to Retrieve CPU SKU & Core Topology	Command to Inspect Cache & Memory Bus
Linux (Ubuntu/Debian)	lscpu grep -E 'model name|cpu cores' /proc/cpuinfo	lscpu -C sudo dmidecode --type memory | grep Speed
macOS (Apple Silicon/Intel)	sysctl -n machdep.cpu.brand_string sysctl hw.perflevel0.physicalcpu	sysctl hw.l1dcachesize hw.l2cachesize system_profiler SPMemoryDataType
Windows (PowerShell / CMD)	Get-CimInstance Win32_Processor | Select Name, NumberOfCores wmic cpu get name,NumberOfCores,NumberOfLogicalProcessors	Get-CimInstance Win32_CacheMemory wmic memorychip get speed,capacity,devicelocator

Bonus Task

Copy-paste this code. Do not change the logic. Run it across thread/process counts from 1 to 32, graph your actual execution times, and answer the autopsy questions.

```
import time
from multiprocessing import Pool
import threading
def cpu_work(n):
    # Pure CPU crunch
    count = 0
    for i in range(n):
        count += i * i
    return count
# Run both experiments with total_work = 50_000_000 split across T workers
# Test T in [1, 2, 4, 8, 16, 32]
```

Deliverables:

1. **The Empirical Curve:** Plot T (workers) on the X-axis vs. Execution Time on the Y-axis for both threading.Thread and multiprocessing.Pool.
2. **The Inflection Cliff:** Identify the exact number of workers where adding more workers makes the program *slower*.
3. **The Architectural Autopsy:**
    - Why does Python's threading curve remain virtually flat or get worse, while multiprocessing scales? (Forces them to articulate the GIL).
    - Look up your laptop's CPU model (e.g., Apple M-series, Intel Core i5/i7, AMD Ryzen). How many *physical performance cores* do you have vs. *logical threads*?
    - Pinpoint the exact point on your graph where physical core saturation occurs, and explain the performance drop when $T > \text{physical cores}$ using context-switching overhead.